Inception Phase – PECI

Dinis Oliveira, André Silva, Rafael Ferreira, Guilherme Goulart, Adriana Rocha

1. Context

Modern datacenters are highly dynamic environments built on:

- Virtualization (VMs)
- Containers (e.g., Kubernetes-based workloads)
- Software-defined networking (SDN)
- Cloud-native microservices architectures

Services are elastic, distributed, and continuously scaled. However, this dynamism introduces:

- Performance variability
- Resource contention
- Cascading failures
- Limited observability across layers (application ↔ OS ↔ network ↔ hardware)

Traditional monitoring solutions are often:

- Reactive instead of proactive
- Focused on isolated metrics (CPU, RAM)
- Not integrated with control or mitigation mechanisms

There is a growing need for **intelligent, automated observability and control systems** that can:

- Detect anomalies early
- Correlate multi-layer metrics
- Support automated remediation

2. Problem

In dynamic datacenters:

1. Performance degradation is often detected too late.

2. Root-cause analysis is complex due to:
    a. Multi-layer architectures
    b. Distributed services
    c. Ephemeral workloads
3. Monitoring systems generate large volumes of metrics and logs but:
    a. Lack intelligent correlation
    b. Do not provide predictive insights
4. Fault injection and stress testing are rarely integrated into continuous monitoring frameworks.

Core problem:
How can we design an intelligent monitoring and control framework capable of detecting, analyzing, and mitigating performance degradation in dynamic datacenter environments?

3. Goals

General Goal

Design and implement a prototype framework for intelligent monitoring and performance control in dynamic datacenters.

Specific Goals

1. Design a multi-layer observability architecture.
2. Integrate real-time metrics collection.
3. Implement fault injection and stress testing.
4. Apply data analysis / ML techniques for:
    a. Anomaly detection
    b. Performance prediction
    c. Bottleneck identification
5. Provide a dashboard for visualization and interaction.
6. (Optional advanced goal) Implement automated mitigation strategies.

4. Tasks (Draft)

Phase 1 – Requirements and Architecture

- Literature review
- Define use cases:
  - CPU saturation
  - Memory leak/usage
  - Network congestion
  - Pod crash loops
- Define system architecture:
  - Monitoring agents
  - Central collector
  - Storage backend
  - Analytics engine
  - Dashboard

Deliverable:

- Architecture document
- Technology selection justification

Phase 2 – Infrastructure Setup

- Deploy experimental environment:
  - Local cluster (e.g., Kubernetes)
  - Virtualized environment
- Deploy monitoring stack:
  - Metrics collection
  - Log aggregation
  - Tracing

Deliverable:

- Running testbed

Phase 3 – Observability Implementation

- Collect:
  - CPU, memory, I/O
  - Network metrics
  - Container-level metrics
  - Application-level metrics
- Centralize and structure data

Deliverable:

- Unified observability pipeline

Phase 4 – Fault Injection & Stress Testing

- Introduce controlled failures:
  - Resource exhaustion
  - Network delay
  - Service crashes
- Measure system response

Deliverable:

- Dataset of fault scenarios

Phase 5 – Intelligent Analysis

- Implement:
  - Statistical anomaly detection
  - ML-based classification/regression
- Evaluate detection accuracy

Deliverable:

- Evaluation report
- Comparative metrics (precision, recall, latency)

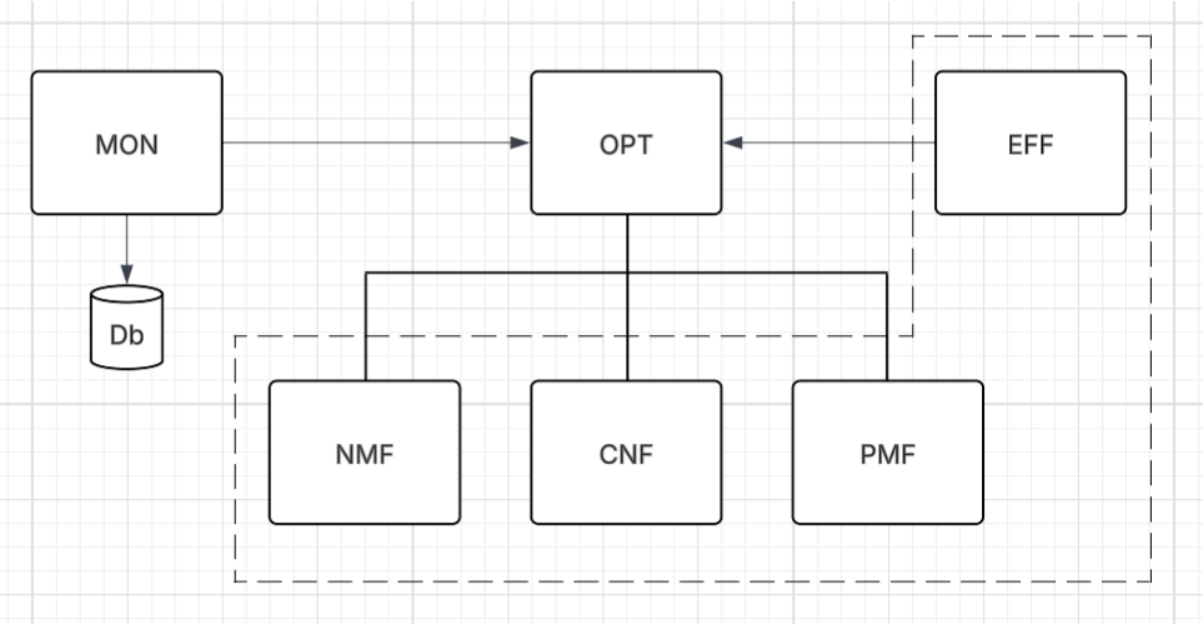
Phase 6 – Visualization & Control

- Develop dashboard:
  - System state overview
  - Alerting
  - Historical analysis
- (Optional) Implement automated scaling or mitigation actions

Deliverable:

- Functional prototype

Global System Overview



The system implements a **closed-loop intelligent monitoring and control architecture** for dynamic datacenters. It integrates observability, analysis, and infrastructure control to detect and mitigate performance degradation in real time.

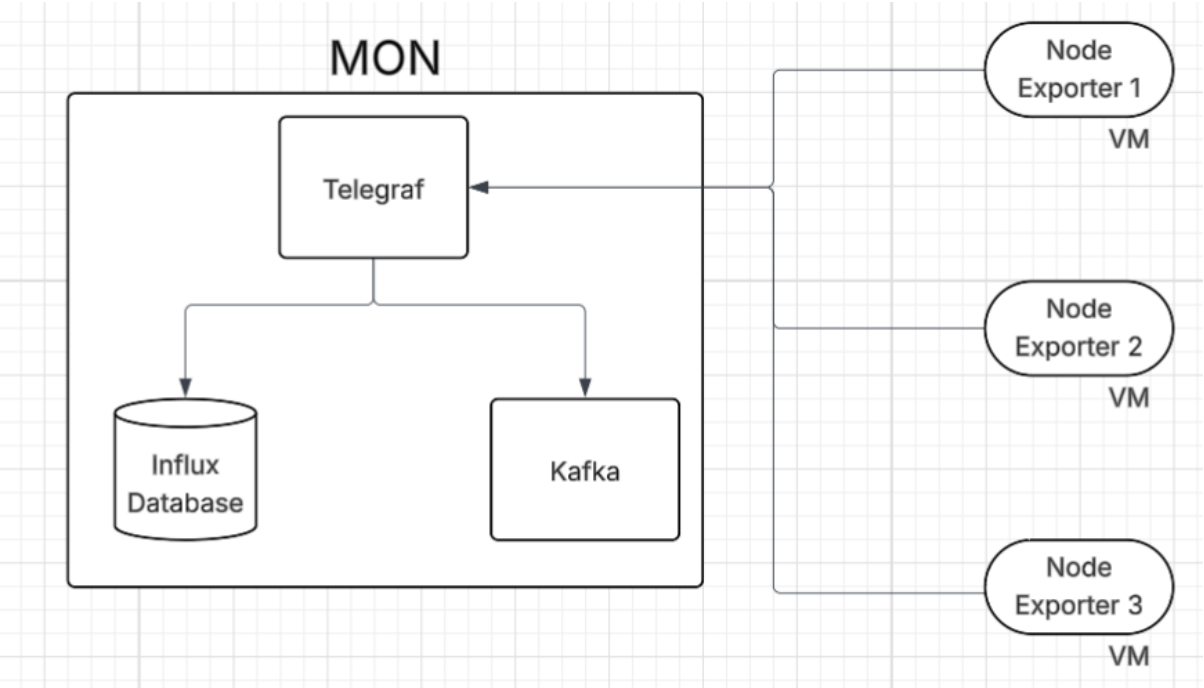
The architecture is composed of:

- **MON (Monitoring)** – Collects telemetry (CPU, memory, network, containers, applications) and stores/streams the data.
  This module is the main focus of the project.

- **OPT (Optimization Engine)** – Analyzes monitoring data, detects anomalies or bottlenecks, and generates optimization decisions.
- **EFF (Efficiency Module)** – Applies resource and energy optimization policies in coordination with OPT.
- **Management Layer (NMF, CNF, PMF)** – Executes control actions:
    - **NMF**: Network control
    - **CMF**: Compute resource management
    - **PMF**: Platform orchestration
- **Db** – Stores historical metrics and analysis data.

The system operates as a feedback loop: infrastructure metrics are collected by MON, analyzed by OPT, translated into control actions, and enforced by the management layer, enabling adaptive and intelligent datacenter operation.

MON Module – Architecture Details



Node Exporters (VMs)

Estes recolhem métricas como:

- CPU usage
- Memory usage
- Disk I/O
- Network throughput

Telegraf

Agente agregador.

- Recolhe métricas dos Node Exporters
- Normaliza dados
- Envia para:
    - InfluxDB (armazenamento)
    - Kafka (streaming em tempo real)

InfluxDB

Base de dados time-series, serve para:

- Armazenamento histórico
- Dashboards
- Análise posterior
- Treino de modelos ML

Kafka

Plataforma de streaming, permite:

- Processamento em tempo real
- Comunicação com o módulo OPT
- Event-driven optimization

5. Expected Results

By the end of the project:

1. Experimental validation with fault injection scenarios.
2. Performance metrics:
    a. Detection latency
    b. Accuracy
    c. Overhead introduced by monitoring
3. A functional prototype of a monitoring and control system.
4. Documentation and technical report.

Optional high-value result:

- Demonstration of predictive maintenance capability.

6. Related Work and Technologies

Monitoring & Observability

- Prometheus
- Grafana
- ELK stack
- OpenTelemetry

Virtualization & Orchestration

- Kubernetes
- Docker
- OpenStack (optional)
- VMware (if lab-supported)

Fault Injection

- Chaos Engineering
- Chaos Mesh
- Stress-ng

Data Analysis & ML

- Python (Pandas, Scikit-learn)
- PyTorch / TensorFlow (if deep learning is used)
- Time-series anomaly detection methods